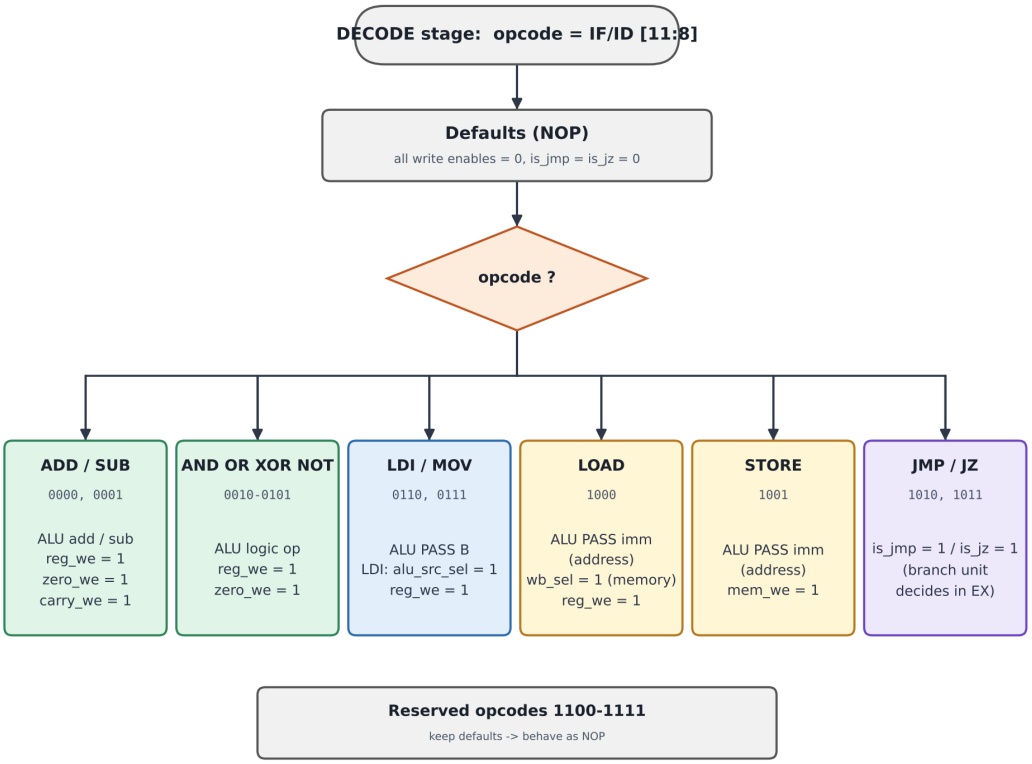
Code walkthrough

control_unit.v

Control Unit

What it does	Decodes instructions and decides jumps
Where it works	ID stage and EX stage
Size	214 lines, explained in 17 small parts
Modules in this file	instruction_decoder, branch_unit, control_unit
Calls (uses)	only modules inside this same file: instruction_decoder, branch_unit
Called by	cpu_top.v - instance CU tb_control_unit.v - instance DUT

The control unit has two working parts: the instruction decoder (ID stage) turns an opcode into control signals, and the branch unit (EX stage) decides whether a jump is taken. A third module joins them under one name.



What the instruction decoder does for each group of opcodes.

Map of the file

Part	Lines	What this part is
1	1-20	Header comment
2	23-38	Decoder: ports
3	39-52	Decoder: names for the opcodes
4	54-61	Decoder: names for the ALU codes
5	63-73	Decoder Step 1: default = do nothing
6	75-88	Decoder Step 2: ADD and SUB
7	89-108	Decoder: AND, OR, XOR, NOT
8	109-117	Decoder: LDI and MOV
9	118-128	Decoder: LOAD and STORE
10	129-134	Decoder: JMP and JZ
11	135-140	Decoder: reserved opcodes
12	143-154	Branch unit: ports
13	155-162	Branch unit: the decision
14	164-165	Branch unit: flush
15	168-191	control_unit: ports
16	192-204	control_unit: the decoder instance
17	206-214	control_unit: the branch unit instance

How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-20)

control_unit.v lines 1-20

```
 1  // ======================================================================
 2  // FILE: control_unit.v
 3  // Hardwired control unit for the 3-stage pipeline.
 4  //
 5  // Interface (as in the project plan):
 6  //   opcode, flags, alu_ctrl, reg_we, mux selects, PC controls
 7  //
 8  // This file contains 3 modules:
 9  //   1. instruction_decoder : DECODE stage - turns the opcode into
10  //                            control signals (pure logic, no clock)
11  //   2. branch_unit         : EXECUTE stage - decides if a jump is taken,
12  //                            and if so asks the pipeline to flush
13  //   3. control_unit        : joins the two modules above
14  //
15  // CONNECTIONS
16  //   Calls       : only modules inside this same file:
17  //                 instruction_decoder, branch_unit
18  //   Called by   : cpu_top.v - instance CU
19  //                 tb_control_unit.v - instance DUT
20  // ======================================================================
```

- Names the three modules and which pipeline stage each one works in.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Decoder: ports (lines 23-38)

control_unit.v lines 23-38

```
23  // ----------------------------------------------------------------------
24  // MODULE 1: instruction_decoder  (used in the DECODE stage)
25  // Turns ON only the signals an instruction needs.
26  // ----------------------------------------------------------------------
27  module instruction_decoder (
28      input      [3:0] opcode,      // instruction bits [11:8]
29      output reg [2:0] alu_ctrl,    // ALU operation
30      output reg       alu_src_sel, // operand MUX: 0 = Rs, 1 = immediate
31      output reg       wb_sel,      // write-back MUX: 0 = ALU, 1 = memory
32      output reg       reg_we,      // 1 = write register Rd
33      output reg       mem_we,      // 1 = write data memory
34      output reg       zero_we,     // 1 = update Zero flag
35      output reg       carry_we,    // 1 = update Carry flag
36      output reg       is_jmp,      // 1 = this is JMP
37      output reg       is_jz        // 1 = this is JZ
38  );
```

- One input: the 4-bit opcode (bits 11:8 of the instruction in IF/ID).
- Nine outputs. Each is a switch for one part of the datapath: alu_ctrl (ALU), alu_src_sel (operand MUX), wb_sel (write-back MUX), reg_we (register file), mem_we (data memory), zero_we and carry_we (flags), is_jmp and is_jz (branch unit).

Part 3 · Decoder: names for the opcodes (lines 39-52)

control_unit.v lines 39-52

```
39      // ---- opcodes (project plan, Section 4) ----
40      localparam ADD   = 4'b0000;
41      localparam SUB   = 4'b0001;
42      localparam AND   = 4'b0010;
43      localparam OR    = 4'b0011;
44      localparam XOR   = 4'b0100;
45      localparam NOT   = 4'b0101;
46      localparam LDI   = 4'b0110;
47      localparam MOV   = 4'b0111;
48      localparam LOAD  = 4'b1000;
49      localparam STORE = 4'b1001;
50      localparam JMP   = 4'b1010;
51      localparam JZ    = 4'b1011;
52      // 1100 - 1111 are reserved: they do nothing (NOP)
```

`localparam` gives a readable name to each 4-bit opcode from the project plan. Writing `ADD:` is clearer than `4'b0000:`. These lines create no hardware. Opcodes 1100-1111 are reserved.

Part 4 · Decoder: names for the ALU codes (lines 54-61)

control_unit.v lines 54-61

```
54      // ---- ALU control codes (must match alu.v) ----
55      localparam ALU_ADD  = 3'd0;
56      localparam ALU_SUB  = 3'd1;
57      localparam ALU_AND  = 3'd2;
58      localparam ALU_OR   = 3'd3;
59      localparam ALU_XOR  = 3'd4;
60      localparam ALU_NOT  = 3'd5;
61      localparam ALU_PASS = 3'd6;
```

The same idea for the 3-bit ALU operation codes. They must match the codes used inside alu.v.

Part 5 · Decoder Step 1: default = do nothing (lines 63-73)

control_unit.v lines 63-73

```
63      always @(*) begin
64          // ---- Step 1: default = do nothing ----
65          alu_ctrl    = ALU_ADD;
66          alu_src_sel = 1'b0;
67          wb_sel      = 1'b0;
68          reg_we      = 1'b0;
69          mem_we      = 1'b0;
70          zero_we     = 1'b0;
71          carry_we    = 1'b0;
72          is_jmp      = 1'b0;
73          is_jz       = 1'b0;
```

- `always @(*)` = combinational logic: the outputs follow the opcode immediately.
- Every output is first set to 0 ("do nothing"). This guarantees every output always has a value (no latches), and any opcode not listed later - the reserved ones - behaves as a NOP.

Part 6 · Decoder Step 2: ADD and SUB (lines 75-88)

control_unit.v lines 75-88

```
 75          // ---- Step 2: turn on what the opcode needs ----
 76          case (opcode)
 77              ADD: begin                    // Rd = Rd + Rs
 78                  alu_ctrl = ALU_ADD;
 79                  reg_we   = 1'b1;
 80                  zero_we  = 1'b1;
 81                  carry_we = 1'b1;
 82              end
 83              SUB: begin                    // Rd = Rd - Rs
 84                  alu_ctrl = ALU_SUB;
 85                  reg_we   = 1'b1;
 86                  zero_we  = 1'b1;
 87                  carry_we = 1'b1;
 88              end
```

- The case statement looks at the opcode and turns on only the signals needed.
- ADD and SUB: set the ALU operation, write the result into Rd (reg_we), and update both flags (zero_we, carry_we).

Part 7 · Decoder: AND, OR, XOR, NOT (lines 89-108)

control_unit.v lines 89-108

```
 89              AND: begin                    // Rd = Rd AND Rs
 90                  alu_ctrl = ALU_AND;
 91                  reg_we   = 1'b1;
 92                  zero_we  = 1'b1;
 93              end
 94              OR: begin                     // Rd = Rd OR Rs
 95                  alu_ctrl = ALU_OR;
 96                  reg_we   = 1'b1;
 97                  zero_we  = 1'b1;
 98              end
 99              XOR: begin                    // Rd = Rd XOR Rs
100                  alu_ctrl = ALU_XOR;
101                  reg_we   = 1'b1;
102                  zero_we  = 1'b1;
103              end
104              NOT: begin                    // Rd = NOT Rd
105                  alu_ctrl = ALU_NOT;
106                  reg_we   = 1'b1;
107                  zero_we  = 1'b1;
108              end
```

Same pattern, but only `zero_we` is turned on: logic operations update Zero and keep the old Carry.

Part 8 · Decoder: LDI and MOV (lines 109-117)

control_unit.v lines 109-117

```
109             LDI: begin                      // Rd = immediate
110                 alu_ctrl    = ALU_PASS;     // ALU passes operand B
111                 alu_src_sel = 1'b1;         // operand B = immediate
112                 reg_we      = 1'b1;
113             end
114             MOV: begin                      // Rd = Rs
115                 alu_ctrl = ALU_PASS;        // ALU passes operand B = Rs
116                 reg_we   = 1'b1;
117             end
```

- Both use the ALU's PASS operation, which copies input B to the output.
- LDI also sets `alu_src_sel = 1`, so B is the immediate number from the instruction. MOV leaves it at 0, so B is register Rs.
- Flags are not changed.

Part 9 · Decoder: LOAD and STORE (lines 118-128)

control_unit.v lines 118-128

```
118             LOAD: begin                     // Rd = data_memory[addr]
119                 alu_ctrl    = ALU_PASS;     // ALU passes addr to memory
120                 alu_src_sel = 1'b1;
121                 wb_sel      = 1'b1;         // write memory data into Rd
122                 reg_we      = 1'b1;
123             end
124             STORE: begin                    // data_memory[addr] = Rd
125                 alu_ctrl    = ALU_PASS;     // ALU passes addr to memory
126                 alu_src_sel = 1'b1;
127                 mem_we      = 1'b1;
128             end
```

- Both send the immediate (the memory address) through the ALU with PASS and alu_src_sel = 1. The ALU output becomes the data-memory address.
- LOAD: `wb_sel = 1` (write the memory value) and reg_we = 1.
- STORE: `mem_we = 1` (write Rd into memory); no register is written.

Part 10 · Decoder: JMP and JZ (lines 129-134)

control_unit.v lines 129-134

```
129             JMP: begin                      // PC = addr
130                 is_jmp = 1'b1;
131             end
132             JZ: begin                       // if Zero = 1, PC = addr
133                 is_jz = 1'b1;
134             end
```

The decoder only marks the instruction as a jump (is_jmp or is_jz). It does not jump: the branch unit makes that decision one clock later in the EX stage, when the Zero flag is up to date.

Part 11 · Decoder: reserved opcodes (lines 135-140)

control_unit.v lines 135-140

```
135              default: begin
136                  // reserved opcodes 1100-1111: do nothing
137              end
138          endcase
139      end
140  endmodule
```

Opcodes 1100-1111 land here and keep all the Step 1 defaults, so they do nothing.

Part 12 · Branch unit: ports (lines 143-154)

control_unit.v lines 143-154

```
143  // ----------------------------------------------------------------------
144  // MODULE 2: branch_unit  (used in the EXECUTE stage)
145  // pc_sel = 1 means "jump": the PC loads the jump address, and the two
146  // younger instructions already in IF and ID are flushed (made NOP).
147  // ----------------------------------------------------------------------
148  module branch_unit (
149      input      is_jmp,        // instruction in EX is JMP
150      input      is_jz,         // instruction in EX is JZ
151      input      zero_flag,     // saved Zero flag
152      output reg pc_sel,        // PC MUX: 0 = PC + 1, 1 = jump address
153      output     flush          // 1 = empty the IF/ID and ID/EX registers
154  );
```

- Inputs describe the instruction now in EX: is_jmp, is_jz, plus the saved Zero flag.
- Outputs: `pc_sel` (1 = PC loads the jump address) and `flush` (1 = empty the pipeline registers).

Part 13 · Branch unit: the decision (lines 155-162)

control_unit.v lines 155-162

```
155      always @(*) begin
156          if (is_jmp)
157              pc_sel = 1'b1;                 // JMP always jumps
158          else if (is_jz && zero_flag)
159              pc_sel = 1'b1;                 // JZ jumps only when Zero = 1
160          else
161              pc_sel = 1'b0;                 // otherwise: next address
162      end
```

- JMP: always jump.
- JZ: jump only if Zero = 1.
- Anything else: no jump, pc_sel = 0 (PC + 1).

Part 14 · Branch unit: flush (lines 164-165)

control_unit.v lines 164-165

```
164      assign flush = pc_sel;                 // jump taken -> flush
165  endmodule
```

When a jump is taken, the two instructions behind it (already in IF and ID) are the wrong ones, so they must be thrown away. That is why flush is simply equal to pc_sel.

Part 15 · control_unit: ports (lines 168-191)

control_unit.v lines 168-191

```
168  // ------------------------------------------------------------------
169  // MODULE 3: control_unit (instruction_decoder + branch_unit)
170  // The decoder works on the instruction in DECODE; the branch unit
171  // works on the instruction in EXECUTE.
172  // ------------------------------------------------------------------
173  module control_unit (
174      // ---- DECODE stage ----
175      input      [3:0] opcode,
176      output     [2:0] alu_ctrl,
177      output           alu_src_sel,
178      output           wb_sel,
179      output           reg_we,
180      output           mem_we,
181      output           zero_we,
182      output           carry_we,
183      output           is_jmp,
184      output           is_jz,
185      // ---- EXECUTE stage ----
186      input            ex_is_jmp,
187      input            ex_is_jz,
188      input            zero_flag,
189      output           pc_sel,
190      output           flush
191  );
```

The decoder ports connect to the ID stage (opcode in, control signals out). The branch ports connect to the EX stage (ex_is_jmp, ex_is_jz and zero_flag in; pc_sel and flush out).

Part 16 · control_unit: the decoder instance (lines 192-204)

control_unit.v lines 192-204

```
192      // CALLS: module instruction_decoder (MODULE 1 in this same file, control_unit.v)
193      instruction_decoder DECODER (
194          .opcode      (opcode),
195          .alu_ctrl    (alu_ctrl),
196          .alu_src_sel (alu_src_sel),
197          .wb_sel      (wb_sel),
198          .reg_we      (reg_we),
199          .mem_we      (mem_we),
200          .zero_we     (zero_we),
201          .carry_we    (carry_we),
202          .is_jmp      (is_jmp),
203          .is_jz       (is_jz)
204      );
```

Places one decoder and connects its ports straight to the control_unit ports with the same names.

→ This part **calls module** `instruction_decoder`, which is MODULE 1 in this same file (see the earlier parts of this document).

Part 17 · control_unit: the branch unit instance (lines 206-214)

control_unit.v lines 206-214

```
206      // CALLS: module branch_unit (MODULE 2 in this same file, control_unit.v)
207      branch_unit BRANCH (
208          .is_jmp    (ex_is_jmp),
209          .is_jz     (ex_is_jz),
210          .zero_flag (zero_flag),
211          .pc_sel    (pc_sel),
212          .flush     (flush)
213      );
214  endmodule
```

Places one branch unit. Note `.is_jmp(ex_is_jmp)`: the branch unit sees the jump marker that was made one clock earlier and has travelled through the ID/EX register.

→ This part **calls module** `branch_unit`, which is MODULE 2 in this same file (see the earlier parts of this document).

> **In short:** Decoder (ID) decides what an instruction does; branch unit (EX) decides whether to jump and flush.